Erőforrásra várakozás: a processzor arra vár, hogy egy erőforrás felszabaduljon, és annak felhasználásával folytathassa a feldolgozást. Itt beszélhetünk elvehető (megszakítható) – pl. memória –, illetve nem elvehető (nem megszakítható) – pl. optikai meghajtó – erőforrásokról, aszerint, hogy az a folyamattól mindenféle következmény nélkül megvonható-e. Valamint vannak felcserélhető erőforrások is, ahol az egyik erőforrás helyettesíthető egy másikkal.

Halálos ölelés (deadlock): Folyamatok egy halmaza akkor van ~ben, ha a halmaz minden folyamata olyan eseményre vár, amelyet csak egy másik, ugyancsak halmazbeli (várakozó) folyamat tudna előidézni (Az esemény többnyire egy erőforrás felszabadulását jelenti).

Kialakulásának szükséges feltételei: akkor jöhet létre, ha az alábbi négy feltétel egyszerre teljesül:

- Kölcsönös kizárás: Egy erőforrást egyszerre legfeljebb egy folyamat használhat.
- Foglalva várakozás: Van olyan folyamat, amely lefoglalva tart erőforrásokat, miközben más erőforrásokat kér (és azokra várakozik).
- Nem elvehető erőforrások vannak a rendszerben: Az erőforrást a folyamat csak önszántából szabadíthatja fel.
- Körkörös várakozás: Létezik egy olyan {P0, P1, ..., Pn } sorozat, amelyben P0 egy P1 által lefoglalva tartott erőforrásra vár, Pi egy Pi+1 által foglaltra, végül Pn pedig egy P0 által foglaltra vár.

Elkerülése: A halálos ölelés elkerülésével kapcsolatban alapvetően 4 alapvető stratégia alakítható ki:

- *Figyelmen kívül hagyjuk* (struccpolitika): Legolcsóbb megoldás. Bizonyos rendszereknek ez sajnos nem megengedhető. Mérlegelni kell a probléma súlyát és a megoldás árát.
- *Detektáljuk és közbelépünk*: Ha deadlock lép fel, vegyük észre és tegyünk valamit, hogy feloldjuk Ha minden erőforrásból csak 1 példány van, akkor detektálható gráfkereső algoritmusokkal, ha van, amelyikből több, akkor mátrixalgebrát használó módszerekkel. Deadlock felismerésekor lehetséges eljárások:
    - *Erőforrás megvonása*: a folyamatot blokkoljuk, az erőforrást visszavonjuk, megvárjuk, amíg a másik folyamat végez, az erőforrást viszaadjuk, majd a folyamat blokkolását feloldjuk
    - *Visszagörgetés*: a folyamat teljes állapotát időről időre elmentjük és szükség esetén visszatöltjük.
    - *Megszakítás*: a deadlockban található folyamatok némelyikét megszakítjuk. Ügyelnünk kell, hogy olyan folyamatot szakítsunk meg, amely újraindítható és az eredmény pótolható.
- *Elkerülés, az erőforrások körültekintő kiosztásával*: Mivel az OS osztja ki az erőforrásokat, képes lehet arra, hogy az erőforrások odaítélését elhalassza, ha úgy találja, hogy a deadlock alakulhat ki. Bonyolult algoritmusokat igényel.
- *Megelőzés, a kialakulásához szükséges 4 feltétel valamelyikének kizárásával*:
    - *kölcsönös kizárás feltétele*: eliminálható, ha az erőforrást egy időben több folyamat által használhatóvá tesszük, azaz megszüntetjük a kölcsönös kizárás szükségességét.
    - *foglalva várakozás állapot*: lehetetlenné tehető, ha kötelezővé tesszük a folyamatnak, hogy előbb az összes erőforrást lefoglalja, csak utána végezze el a munkát. Nem szerencsés (feleslegesen fogja az erőforrásokat).
    - *megszakíthatatlanság*: ha vmelyik szükséges erőforrás nem áll rendelkezésre, akkor a többit is elvesszük tőle, és a folyamat várakozik. Akkor futhat tovább, ha az összes erőforrásigényét egyszerre lehet kielégíteni.
    - *körkörösség* feltétele több módszer segítségével is kizárható a rendszerből:
        - Minden folyamat csak egy erőforrást tarthat magánál. Ez sokszor nem tartható be.
        - A folyamatok csak bizonyos sorrendben foglalhatják le az erőforrásokat. Ezzel az a probléma, hogy nem minden erőforrás számozható meg.
        - A folyamat csak a lefoglalt erőforrások közül a legnagyobb sorszámú erőforrásnál magasabb sorszámút kérhet.

TeX: dokumentációkészítő rendszer

LaTeX: a TeX nyelvén megírt makrórendszer. Parancssoros eszköz.

Működése: a szöveges kódot .tex fájlban megírjuk, majd (la)tex <file> paranccsal <file>.dvi formátumra alakítjuk.

Mintakód:

```
\documentclass {article}                 (dok. típus, lehet még book, letter stb.)
\usepackage[utf8] {inputenc}             (karakterkódolás)
\usepackage[english,magyar] {babel}      (magyar elválasztás)
\begin{document}                         (dok. kezdete)
  …
\end{document}                           (dok. vége)
```

Dokumentumtagolás: \part{cím} … \section {cím} … \subsection{cím} … \subsubsection {cím} … \paragraph{cím} … subparagraph {cím}

Parancs: szabványos forma: *parancs {paraméterek}[opcionális paraméterek]*. Példák:

\footnote{…} *(lábjegyzet)*

\frac{alap}{kitevő} *(hatványozás)*

AWK: egy szövegfeldolgozó eszköz, interpretált makróprocesszor

- Minden sort elolvas és feltételekre illeszt
- Az illeszkedő sorokra tevékenységeket hív meg
- (egy tevékenység lehet több soros is)

- Az input sorokat mezőkre töri szét
  - $0 Az alma piros
  - $1 Az
  - $2 alma
  - $3 piros

AWK indítása:

```
awk –f parancsfile inputfile
```

- Hello világ program: {print "Hello világ!"}
- Mezők kiíratása: {print $1 "\t" $2}
- BEGIN Előfeldolgozás (a legelső sor előtt)
  END Utófeldolgozás (a legutolsó sor után)

Reguláris kifejezések és használatuk:

- () zárójelezés
- ^ sorkezdet, $ sorvég
- . bármilyen karakter, nem sorvégjel
- [abc] a vagy b vagy c karakterek
- [^abc] nem a, nem b, nem c karakterek, nem sorvégjel
- r1|r2 alternatívák
- r+ egy vagy több előfordulás
- r* nulla vagy több előfordulás
- r? nulla vagy egy előfordulás
- E-betűt tartalmazó nevek sorának kiíratása

```
$1~/^E/ {print $0}
```

- A-betűvel végződő nevek sorának kiíratása

```
$1~/a$/ {print $0}
```

- Diszjunkció

```
$1~/^E/ || $1~/a$/ {print $0}
```

- Nem számjegy

```
$0!~/^[0-9]/{print $0}
```